SIDE A · THEORY 1 Asymptotics · 2 Recurrences · 3 Binary search & divide and conquer · 4 Sorting & selection | **SIDE B** 5 Randomness · 6 Heaps · 7 Lower bounds & linear sorting · 8 Exam protocol. Blue box = meaning, bordered = formal, green = template to copy, yellow = result to quote, red = pitfalls.

1 · ASYMPTOTICS (Q1 on every past paper)

IN WORDS How fast do two functions grow for huge n? Constant factors and small n do not count. O = "grows no faster than" (ceiling). Ω = "grows at least as fast" (floor). Θ = "same growth rate". Example: $5n + 3$ is $O(n^2)$; $3n^2 + 7n$ is $\Theta(n^2)$.

FORMAL (write exactly) $f = O(g)$: there exist constants $c > 0$ and n_0 such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$. $f = \Omega(g)$: $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$. $f = \Theta(g)$: both (prove O and Ω separately). Worst case $\Theta(g) = O(g)$ on every input AND some input family costs $\Omega(g)$.

Growth ladder, slow to fast

$1 < \log\log n < \log n < (\log n)^k < n^{0.1} < n < n \log n < n^2 < n^k < 2^n < n! < n^n$
Any power of log < any polynomial < any exponential. A huge constant is $\Theta(1)$.

Rewrite rules

- Keep only the biggest term: $n^3 + 1000n^{2.9} = \Theta(n^3)$.
- Log base never matters: $\log_{10} n = \Theta(\log_2 n)$. Exponent base does: $3^n \gg 2^n$.
- Exponent: added constant is same class, $2^{n+5} = 32 \cdot 2^n = \Theta(2^n)$; multiplied constant is different, $2^{2n} = 4^n \ne \Theta(2^n)$.
- $\log(n^c) = c \log n = \Theta(\log n)$. $\log(2^n) = n$. $\log(n!) = \Theta(n \log n)$.
- $a^{\log_b n} = n^{\log_b a}$: $4^{\log_2 n} = n^2$, $3^{\log_2 n} = n^{1.585}$, $2^{\log n} = n$.
- Weird functions: write each as $2^{(\text{something})}$ and compare the somethings; $(\log n)^{\log n} = n^{\log\log n}$. Or substitute $m = \log n$: $\sqrt{(\log n)}$ vs $\log \sqrt{n} = \tfrac{1}{2} \log n \Rightarrow \Omega$ only.
- $n! \le n^n$, $n! \ge (n/2)^{n/2}$, $n! = \Theta(\sqrt{n}\,(n/e)^n)$. $n^{1/\log n} = 2$. $\max(f, g) = \Theta(f + g)$.

Sums (do by hand)

- $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$. $\sum i^c = \Theta(n^{c+1})$. Upper: n × biggest term. Lower: **half trick**, keep the top half, each $\ge (n/2)^c$.
- $H_n = 1 + \tfrac{1}{2} + \tfrac{1}{3} + \dots + 1/n = \Theta(\log n)$, about ln n. Blocks of 1, 2, 4, 8 terms each sum to between ½ and 1.
- Geometric $1 + r + r^2 + \dots$: $r > 1 \Rightarrow \Theta(\text{last term})$; $r < 1 \Rightarrow \Theta(\text{first term})$; $r = 1 \Rightarrow$ count the terms.

TEMPLATE A / B / U: ① simplify both to one Θ term (solve any recurrence or sum first). ② different classes ⇒ the smaller is O of the bigger (A = O(B) or A = Ω(B)). ③ same class ⇒ Θ. ④ one-sided bound given (A = O(n²), B = Ω(n)): ask "could the hidden true value flip the order?" yes ⇒ U, no ⇒ answer. ⑤ O vs O, Ω vs Ω ⇒ U.

RESULT Reused rows: $n \log n$ vs $n^2/\log n \Rightarrow O \cdot \log(n!)$ vs $n \log n \Rightarrow \Theta \cdot 2^n$ vs $2^{n/2} \Rightarrow \Omega \cdot n^{0.1}$ vs $(\log n)^{10} \Rightarrow \Omega \cdot n^{1/2}$ vs $n^{\sin n} \Rightarrow U \cdot 2^n$ vs $n! \Rightarrow O \cdot n^{\log\log n}$ vs $2^{(\log n)^2} \Rightarrow O \cdot n \cdot 2^n$ vs $3^n \Rightarrow O$.

2 · RECURRENCES (9 of 11 papers)

IN WORDS A recursive algorithm's time is "time for n = (number of calls) × (time for the smaller size) + (work done in this call)". In $T(n) = a \cdot T(n/b) + f(n)$: a = calls, b = how much smaller, f = split and combine work. Mergesort: two sorts of n/2 plus n merging = $2T(n/2) + n$. Solving = unroll until the base case and add up what you collected.

TEMPLATE Expansion "from first principles" (no Master theorem):
$T(n) = a\,T(n/b) + f(n)$
$= a^2\,T(n/b^2) + a\,f(n/b) + f(n)$
$= a^i\,T(n/b^i) + \sum_{j<i} a^j f(n/b^j)$ ← state this general level i
stop at $n/b^i = 1$, $i = \log_b n$, $a^i = n^{\log_b a}$ leaves
$T(n) = n^{\log_b a} \cdot T(1) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$; evaluate the geometric sum, say which end wins.

Recursion tree table, same thing: level i has a^i subproblems of size n/b^i, work $a^i \cdot f(n/b^i)$ per level; last level $\log_b n$ has $n^{\log_b a}$ leaves each costing $T(1)$.

RESULT Master, $T = aT(n/b) + n^d$: $a > b^d \Rightarrow \Theta(n^{\log_b a})$, leaves win · $a = b^d \Rightarrow \Theta(n^d \log n)$, every level equal · $a < b^d \Rightarrow \Theta(n^d)$, root wins. Extra log in f and $a = b^d \Rightarrow$ one more log power.

Recurrence	Θ	Recurrence	Θ
$T(n/2) + 1$	$\log n$	$T(n/2) + n$	n
$2T(n/2) + 1$	n	$2T(n/2) + n$	$n \log n$
$2T(n/2) + n \log n$	$n \log^2 n$	$2T(n/2) + n^2$	n^2
$3T(n/2) + n$	$n^{1.585}$	$4T(n/2) + n$	n^2
$7T(n/2) + n^2$	$n^{2.807}$	$8T(n/2) + n^2$	n^3
$T(n-1) + n$	n^2	$T(n-1) + 1$	n
$2T(n-1) + 1$	2^n	$T(n/2) + \log n$	$\log^2 n$
$T(\sqrt{n}) + 1$	$\log\log n$	$T(\sqrt{n}) + \log n$	$\log n$
$T(n/4)+T(3n/4)+n$	$n \log n$	$2T(n/4) + \sqrt{n}$	$\sqrt{n} \log n$
$2T(n/2) + n/\log n$	$n \log\log n$	$27T(n/3) + n^2$	n^3

- $T(\sqrt{n})$: set $m = \log n$, $n = 2^m$, $S(m) = S(m/2) + f$. $T(n-1) + f(n)$: just a sum $\sum f(i)$.
- Binary search $T(n/2) + 2$ unrolls to $2 \log n + 1$. Hanoi $2T(n-1) + 1 = 2^n - 1$. Mergesort $2T(n/2) + n = n \log n + n$.

3 · BINARY SEARCH & DIVIDE AND CONQUER

IN WORDS Split into smaller copies of the same problem, solve recursively, combine. Read the target time backwards: O(log n) = halve with O(1) work · O(n log n) = two halves + O(n) combine · $O(\log^2 n)$ = log n binary searches · O(log k) = halve k · O(n log k) = heap of size k or recursion depth log k · O(n) expected = random pivot.

```
BinarySearch(A,l,r,x): if l>r return nil; m=⌊(l+r)/2⌋
  if A[m]=x return m; if x<A[m] go (l,m-1) else (m+1,r)
T=T(n/2)+2 ⇒ O(log n)
LowerBound(x): l=1, r=n+1; while l<r: m=⌊(l+r)/2⌋
  if A[m]<x then l=m+1 else r=m; return l      (first ≥ x;
"≤" gives first > x)
Rotated max k: if A[m]>A[m+1] return m; if A[m]≥A[l] go
right else left
Last 0 in 000111: A[m]=0&A[m+1]=1 ⇒ m; A[m]=0 ⇒ right else
left. k ≪ n:
  gallop i=1,2,4,… until A[i]=1 (log k steps), then search
(i/2,i)  O(log k)
Distinct values (Θ(log n) many): LowerBound of successor
repeatedly  O(log² n)
```

- **Mergesort**: sort halves, Merge Θ(n). Θ(n log n) on every input, stable, O(n) space.
- **Inversions** ($i < j$, $A[i] > A[j]$): Sort-and-Count; in merge, when R[j] is taken add the number of left items still unmerged ($m - l - i + 2$). Θ(n log n).
- **Max subarray**: best in left, best in right, best crossing = best suffix ending at m + best prefix from m+1 (two linear scans). $2T(n/2) + n = \Theta(n \log n)$. Kadane O(n): $V = \max(V + A[i], A[i])$. Stock profit = max subarray of the day-to-day differences.
- **Karatsuba**: $ab = 2^n a_1b_1 + 2^{n/2}(a_1b_0 + a_0b_1) + a_0b_0$ and $a_1b_0 + a_0b_1 = (a_1+a_0)(b_1+b_0) - a_1b_1 - a_0b_0$, so 3 products. $3T(n/2) + n = \Theta(n^{1.585})$; naive 4 products $\Theta(n^2)$. **Strassen**: 7 block products, $7T(n/2) + n^2 = \Theta(n^{2.807})$; naive 8 ⇒ $\Theta(n^3)$.
- **k-th smallest of two sorted arrays** in O(log k): compare A[k/2] with B[k/2], discard the first k/2 of the smaller side, $k \leftarrow k/2$; $k = 1 \Rightarrow$ min of heads. **k equal blocks** O(n log k): Select median Θ(n), partition, recurse both sides, depth log k.

TEMPLATE Design answer: ① idea in two sentences ② "Procedure Name(input) → output" ③ pseudocode or numbered steps ④ black boxes with their cost ("Select Θ(n)", "Merge Θ(n)", "binary search O(log n)") ⑤ recurrence or sum, solved, final Θ ⑥ one line why it is right.

4 · SORTING & SELECTION

```
Partition(A,l,r): x=A[r]; i=l-1; for j=l..r-1: if A[j]≤x:
i++, swap A[i],A[j]
  swap A[i+1],A[r]; return i+1        Θ(n), in place.
A[l..i]≤x | A[i+1..j-1]>x | ? | x
Quicksort(l,r): if l≥r return; m=Partition; QS(l,m-1);
QS(m+1,r)
Select(A,l,r,i): random pivot → swap to A[r]; m=Partition;
k=m-l+1
  i=k ⇒ A[m]; i<k ⇒ Select(l,m-1,i); else
Select(m+1,r,i-k)         O(n) expected
```

RESULT Quicksort Θ(n log n) expected: sort the values $z_1 < \dots < z_n$. z_i, z_j are compared ⇔ the first pivot picked from $\{z_i..z_j\}$ is z_i or $z_j \Rightarrow Pr = 2/(j - i + 1)$. $E = \sum_{i<j} 2/(j-i+1)$; each row $\le 2H_n = O(\log n)$, n rows ⇒ $O(n \log n)$. Worst $\Theta(n^2)$: sorted input with last-element pivot. Best: median pivot, $2T(n/2) + n$.

RESULT Select O(n) expected: pivot is "good" if in the middle half, Pr = ½, a good pivot discards ≥ ¼. After i good pivots size ≤ $(\tfrac{3}{4})^i n$; 2 calls per good pivot on average ⇒ $E \le \sum 2(\tfrac{3}{4})^i n = 8n$.

Sort	Worst	Stable	Space	Use when
Insertion	n^2 (best n)	yes	1	nearly sorted: O(n + inversions)
Mergesort	n log n	yes	n	stability, lists, guarantee
Quicksort	n^2 (exp. n log n)	no	log n	fast in practice
Heapsort	n log n	no	1	in place + guaranteed
Counting	n + k	yes	n + k	integers in [0, k], k = O(n)
Radix	d(n + k)	yes	n + k	integers < n^c: c passes base n

PITFALLS Quicksort and heapsort are not stable · mergesort needs O(n) extra space · "each element ≤ d from its place" ⇒ heap of size d + 1, O(n log d).

5 · RANDOMNESS & EXPECTATION (9 of 11 papers)

IN WORDS Expected value = long-run average. To get "how many X happen on average", never compute it directly: split the count into yes/no questions ("does person i get own hat?"), the average of a yes/no question is just its probability, and the average of a sum is the sum of the averages, always, even when the questions depend on each other. Hats: n people × 1/n = 1.

FORMAL (write exactly) Let X_i = 1 if event i happens, 0 otherwise (indicator). E[X_i] = Pr[X_i = 1]. X = Σ X_i. By linearity of expectation E[X] = Σ E[X_i] = Σ Pr[event i], no independence needed. (E[XY] = E[X]E[Y] only if independent.) Waiting time with success probability p: E = 1/p. Tail sum: E[X] = Σ_{i≥1} Pr[X ≥ i]. Randomized running time = expected time on the worst input.

Problem	X_i = 1 if …	Pr	E[X]
Hat check, own hat	person i gets own	1/n	1
Own hat or better (rank ≤ own)	person i	i/n	(n+1)/2
Guess n cards, no memory	guess i right	1/n	1
Guess with memory / hiring	i-th is best so far	1/i	H_n = Θ(log n)
Hashing, n keys, m slots	y collides with x	1/m	n/m; ops O(1 + n/m)
Birthday, k people, n days	pair (i,j) same	1/n	k(k−1)/(2n); ≥ 1 at k ≈ √(2n)
Coupon, all n types	X_i = boxes in stage i	(n−i)/n ⇒ E n/(n−i)	n H_n = Θ(n log n)
Coupon, first n/2 types	same, i < n/2	≥ ½ ⇒ E ≤ 2	≤ n = Θ(n)
Distinct types after n boxes	type t seen	1 − (1−1/n)^n	n(1 − (1−1/n)^n) ≈ 0.63n
Quicksort comparisons	z_i, z_j compared	2/(j−i+1)	Θ(n log n)
Random-access search [F25]	good call, middle half	½, removes ¼	≤ Σ 2(¾)^i n = O(n)

TEMPLATE Write every time: "Let X_i = 1 if …, else 0. Pr[X_i = 1] = … . X = Σ X_i. By linearity of expectation E[X] = Σ Pr[…] = … ." Then simplify (H_n ≈ ln n, geometric sum).

PITFALLS Self excluded ⇒ (1 − 1/n)^(n−1), not ^n · never assume independence, sums do not need it · give the exact form when asked, Θ otherwise.

6 · HEAPS (drawing + design, near certain)

IN WORDS A min-heap is a binary tree filled level by level, left to right, where every parent ≤ its children, so the smallest is always on top; siblings are in no order. It lives in a plain array, no pointers.

FORMAL (write exactly) H[1..n]: children of i are 2i and 2i+1, parent ⌊i/2⌋, height ⌊$\log_2$ n⌋, min at H[1]. Insert: append at n+1, bubble up while < parent, O(log n). Extract-min: move H[n] to H[1], n−−, sift down swapping with the smaller child, O(log n). Find-min O(1). Delete position p: replace by H[n], bubble up or sift down. Build-heap: sift down i = ⌊n/2⌋ … 1, O(n). Heapsort: build max-heap, n times swap H[1] with H[n], n−−, sift down; O(n log n), in place, not stable.

TEMPLATE Drawing (0.5 off per mistake): after EVERY insert write the array and the tree. Insert 4, 3, 8, 2: [4] · [4,3] → 3 < 4 swap → [3,4] · [3,4,8] stays · [3,4,8,2] → 2 < 4 → [3,2,8,4] → 2 < 3 → [2,3,8,4]. Extract-min: write the root taken, move the last to the root, sift down comparing with the smaller child, one line per swap.

RESULT Build-heap O(n): a node at height h costs O(h), there are ≤ n/2^(h+1) of them, Σ_h h·n/2^(h+1) = (n/2)·Σ h/2^h = (n/2)·2 = O(n). Inserting one by one is Σ log i = Θ(n log n). [F25] merge two heaps under a new root and sift it: same sum, ≤ n + 1.

Design	Idea	Time
k-way merge of N items	min-heap of (value, list, index), size ≤ k; pop min, push next from that list	O(N log k), space O(k)
k-nearly-sorted array	min-heap of the next k + 1 items	O(n log k)
Top-k largest	min-heap of size k, pop when size > k	O(n log k)
Running median	max-heap low half, min-heap high half, sizes differ ≤ 1, rebalance by moving a root	insert O(log n), median O(1)
Sliding window max, width W	max-heap of (value, index); pop roots whose index left the window (lazy). Heap can reach Θ(n) on increasing input. With a position map, delete the leaver at once ⇒ size W	O(n log n) lazy; O(n log W) with map; deque O(n)
Largest ⌈log n⌉ in O(n loglog n) [S26]	n/log n blocks of log n, sort each; heap of block tops; extract-max and push the next from that block, log n times	n loglog n + n + $\log^2$ n
m sets, findmin O(1), add O(log m + log n_i) [S25]	one heap per set + top heap of the m minima with a back pointer to each set's slot; new x < min_i ⇒ overwrite its slot, bubble up updating pointers	as required
d-ary heap [Tut 5]	children d(i−1)+2 … di+1, parent ⌊(j−2)/d⌋+1, height log_d n	insert O(log_d n), extract O(d log_d n)

7 · LOWER BOUNDS & LINEAR-TIME SORTING

IN WORDS Each comparison is a yes/no question; q questions can tell apart at most 2^q cases. If a problem has N possible answers you need 2^q ≥ N, so q ≥ $\log_2$ N. Sorting has n! answers and log(n!) ≈ n log n, so no comparison sort beats n log n. Counting and radix sort never compare keys, so they escape.

TEMPLATE Lower bound, four lines: ① count the distinct outputs N ② "any comparison algorithm is a binary decision tree with ≥ N leaves" ③ "a binary tree of height h has ≤ 2^h leaves, so h ≥ $\log_2$ N" ④ simplify; height = worst-case comparisons.

Problem	Outputs N	Bound
Sort n	n!	log n! = Ω(n log n) (n! ≥ (n/2)^(n/2))
k smallest in order	n!/(n−k)! ≥ (n/2)^k	Ω(k log n)
Merge two sorted n-arrays	C(2n, n)	≈ 2n = Ω(n)
Split into k sorted blocks	n!/((n/k)!)^k	Ω(n log k)

```
CountingSort(A, keys 0..k): C[v]=count of v; prefix sums C[v]+=C[v-1]
  for j=n downto 1: B[C[A[j]]]=A[j]; C[A[j]]--
Θ(n+k), stable (backwards!)
Radix: sort by least significant digit first, stable counting sort per digit
  d digits base k: Θ(d(n+k)); w-bit keys base k: Θ(⌈w/log k⌉(n+k)); best base k = n
```

RESULT Radix is linear ⇔ keys have O(log n) bits ⇔ range is polynomial: [0, n^3) ⇒ 3 passes base n = Θ(n). Range n^(ln n) ⇒ Θ(log n) passes ⇒ Θ(n log n), no better than comparison. Stability is what makes radix correct (induction on digit position); forward scan would reverse ties.

- Fractions p/q with p, q ≤ n in O(n): two distinct ones differ by ≥ $1/n^2$, key ⌊n^2 p/q⌋ ∈ [n, n^3], radix base n. No gcd, no floats.
- Range count [a,b] in O(1): prefix sums C[b] − C[a−1]. Sorted array: LowerBound twice, O(log n).

8 · EXAM PROTOCOL & MATH FACTS

- 2 hours, 45 to 60 points: 2 minutes per point. Order: asymptotics, heap, expectation, designs. No correctness proofs needed. Running time: always one line ("T(n) = 2T(n/2) + n ⇒ Θ(n log n)" or "n inserts × O(log k)").
- Black boxes: any class, homework or tutorial algorithm, named with its cost. A modified one must be explained in full. Define input and output of every subroutine.

PITFALLS Include the leaf level in recursion trees · log means $\log_2$ · "faster" = smaller running time · U only when a one-sided bound truly leaves it open · give both O and Ω when asked for Θ · heap drawing after every single insert · never skip the indicator sentence.

RESULT log(ab) = log a + log b · log(a^b) = b log a · $\log_b a = \log a / \log b$ · n! ≈ √(2πn)(n/e)^n · C(2n,n) ≈ $4^n/\sqrt{\pi n}$ · Σ_{i≤k} $2^i = 2^{k+1} - 1$ · Σ $x^i = 1/(1-x)$ · Σ i $x^i = x/(1-x)^2$ · $(1 - 1/n)^n \to 1/e$.